

PASSWORD RESET · BACKEND SPEC

STATELESS OTP

How a 6-digit reset code is generated, verified, and automatically invalidated — without a single row ever written to a database, cache, or in-memory store.

HMAC-SHA256

RFC 4226 TRUNCATION

ZERO STORAGE

APP/CORE/OTP_SERVICE.PY

CONTENTS

- | | |
|--|---|
| 01 The problem this avoids | 06 The two calls, end to end |
| 02 The formula | 07 Not leaking who has an account |
| 03 From digest to digits | 08 Where it lives in the code |
| 04 Time windows & the grace period | 09 Security properties |
| 05 Self-invalidation for free | 10 Configuration |

01

THE PROBLEM THIS AVOIDS

Most OTP flows keep a copy of the code somewhere — a row in Postgres, a Redis key, an in-process dict — tagged with an expiry, so verification is a lookup: *does a matching, unexpired code exist for this user?* That copy is itself something to protect, expire, and clean up, and if the process restarts or a request lands on a different instance than the one that issued the code, verification breaks.

This backend takes the opposite approach: nothing about the code is ever written down. A code is a pure function of four inputs — a server secret, the user's email, a 5-minute time step, and the user's current password hash — so **verifying** a submitted code just means recomputing the same function and comparing. Anyone holding those four inputs gets the same answer, on any instance, with no shared store required.

02 THE FORMULA

Every code in this system, generated or verified, comes out of one expression:

```
OTP = truncate6( HMAC-SHA256( SERVER_SECRET, email + ":" + time_window +  
":" + password_hash ) )
```

SERVER_SECRET	A ≥256-bit random value read from OTP_SERVER_SECRET — never in source, never logged. It's the one thing an attacker needs and can't derive from anything public; without it, no one outside the server can produce a code that verifies.
email	Binds the code to one account, so the same time step and password hash still produce a different code per user.
time_window	A 5-minute step counter — see §04.
password_hash	The account's current Argon2 hash. It's already sitting in memory for any request that looked the user up; nothing extra is fetched or sent. See §05 for what this buys.

03 FROM DIGEST TO DIGITS

HMAC-SHA256 outputs 32 bytes — not something a person can read back over a phone call. `_dynamic_truncate()` turns that digest into a 6-digit number using the same scheme RFC 4226 defines for HOTP:

- 1 Take the low nibble of the digest's **last byte** as an offset, 0–15.
- 2 Read the **4 bytes** starting at that offset as a big-endian integer, masking off the top bit of the first byte so the result is always positive.
- 3 Take that integer **mod 10^6** and zero-pad to 6 digits — e.g. "042391".

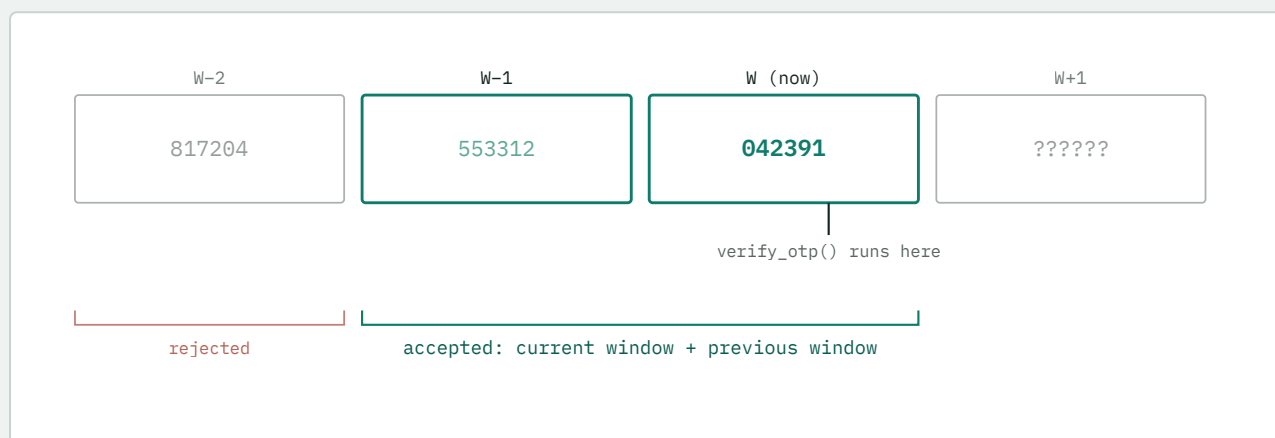


The message string is HMAC-SHA256'd with the server secret, then RFC 4226-style dynamic truncation reads 4 bytes out of the digest — at an offset the digest itself picks — and reduces them to a zero-padded 6-digit code.

Because the offset is derived from the digest, which four of the 32 bytes get used shifts unpredictably from one code to the next — there's no fixed byte range an attacker can focus on.

04 TIME WINDOWS & THE GRACE PERIOD

`time_window = floor(unix_time / 300)` — the clock is chopped into fixed 5-minute steps, and every email + password-hash pair gets a new code each time the step advances. Nothing schedules that change; it falls out of the formula because the window number is itself an input to the hash.



Each 5-minute window hashes to its own code. A verification request checks the code against the current window and the one before it — everything older is rejected, and the next window's code doesn't exist yet.

A code minted a few seconds before a window boundary would otherwise go stale almost immediately, so `verify_otp()` checks the current window *and* the one before it — a single-step grace period. That makes a code's real lifetime somewhere between 5 and 10 minutes depending on where inside its window it was issued, not a flat 5.

CONSTANT-TIME BY CONSTRUCTION

Both windows are checked *unconditionally*, and the comparison uses `hmac.compare_digest` rather than `==` — every submission is compared byte-for-byte in constant time against both candidates, and the loop never returns early on a match. An attacker timing responses can't learn whether a guess matched the current window, the previous one, or neither.

05

SELF-INVALIDATION FOR FREE

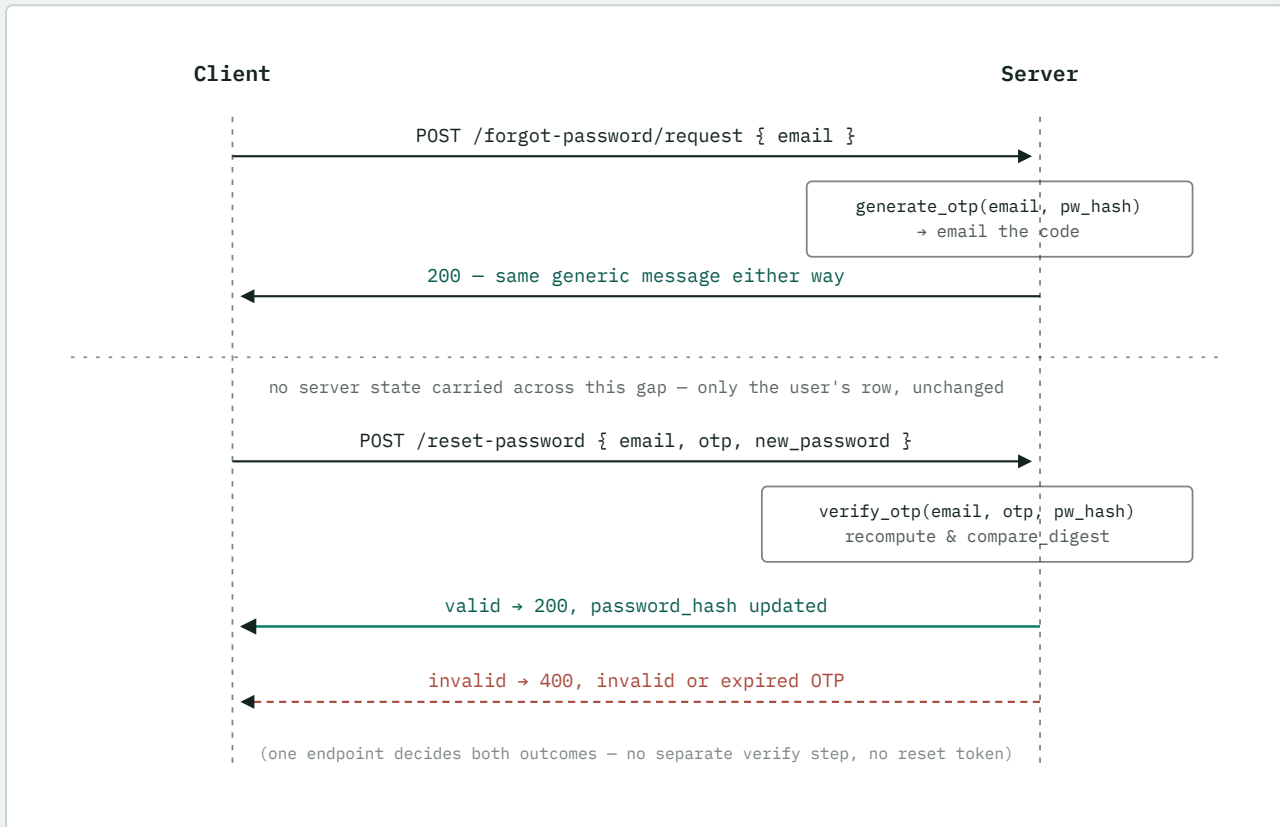
The fourth input to the hash isn't a secret at all — it's the account's own `password_hash`, already loaded into memory for any request that touches this user. Folding it into the HMAC input gets something for free: the moment a password actually changes, `password_hash` changes with it — Argon2 salts every hash uniquely, even for the same plaintext — so every code computed under the old hash, for the current window, the previous one, or any window that ever existed, stops matching.

`AuthService.reset_password()` sets the new hash and commits in the very same call that just verified the old code. There's no revocation list to maintain and no cleanup job to run, because there was never anything to revoke — just a formula whose answer changed underneath it.

06

THE TWO CALLS, END TO END

The whole flow is two HTTP calls. Nothing server-side connects them except the row in `users` — no session, no token, no intermediate record.



Two calls, no state in between: the request step never writes anything, and the reset step both verifies the code and updates the password in one transaction — matched or not, the response is decided entirely by recomputing the formula.

07

NOT LEAKING WHO HAS AN ACCOUNT

`request_otp()` looks the user up first, and only generates and emails a code if a row exists — but the HTTP response is identical either way:

```
{ "message": "If that email is registered, an OTP has been sent to it." }
```

An attacker probing arbitrary addresses can't distinguish "sent" from "no such account" from the response alone — the only difference is whether an email actually goes out, which they can't observe.

08

WHERE IT LIVES IN THE CODE

CONCEPT	SYMBOL	LOCATION
Time-step counter	<code>_current_time_window()</code>	<code>otp_service.py:8</code>
RFC 4226 dynamic truncation	<code>_dynamic_truncate()</code>	<code>otp_service.py:12</code>
HMAC-SHA256 over the message	<code>_compute_otp()</code>	<code>otp_service.py:27</code>
Public: mint a code	<code>generate_otp()</code>	<code>otp_service.py:36</code>
Public: check a submission	<code>verify_otp()</code>	<code>otp_service.py:47</code>
Send + anti-enumeration	<code>AuthService.request_otp()</code>	<code>auth_service.py:156</code>
Verify + update password	<code>AuthService.reset_password()</code>	<code>auth_service.py:175</code>
POST <code>/auth/forgot-password/request</code>	<code>request_otp()</code> route	<code>auth_router.py:120</code>
POST <code>/auth/reset-password</code>	<code>reset_password()</code> route	<code>auth_router.py:136</code>
Secret, window width, digit count	Settings	<code>config.py:31-33</code>

09

SECURITY PROPERTIES

PROPERTY	STATUS	NOTES
High-entropy, non-hardcoded server secret	holds	Read from <code>OTP_SERVER_SECRET</code> ; a validator rejects anything under 32 chars (≈ 256 bits).
Auto-invalidation on password change	holds	<code>password_hash</code> is part of the HMAC input — see §05.
Constant-time comparison	holds	<code>hmac.compare_digest</code> , both windows always checked, no short-circuit.
No account enumeration	holds	Identical response whether or not the email exists — see §07.
Rate limiting on OTP requests / verification	not implemented	Deliberately left out of the current scope — a real deployment should throttle both endpoints per email and per IP before going live.
OTP never logged	partial	Application code never logs a code, but <code>EmailService</code> prints it to stdout when no <code>SMTP_*</code> vars are set — a dev-only fallback that must not run in production.
Delivery over a secure channel	partial	<code>smtpplib</code> calls <code>starttls()</code> when SMTP is configured; without it, delivery falls back to the console log above.

BEFORE PRODUCTION

Two gaps above are load-bearing for a real deployment: add request/verify rate limiting, and make sure `SMTP_HOST`/`SMTP_USER`/`SMTP_PASSWORD`/`SMTP_FROM` are set so the console fallback never fires outside local dev.

10

CONFIGURATION

SETTING	DEFAULT	MEANING
OTP_SERVER_SECRET	required, no default	HMAC key. Must be ≥32 characters; the app refuses to start otherwise.
OTP_WINDOW_SECONDS	300	Width of one time step — how often the code changes, and the unit of the grace period in §04.
OTP_DIGITS	6	Length of the emitted code after truncation.

FastAPI backend · async SQLAlchemy · Argon2 password hashing. This document covers only the forgot-password OTP path — the surrounding register / login / refresh flows are unrelated to it.